

Queueing System Simulation

Code Explanation Guide

Bruno Zomignani Perciani

1. Overview

This MATLAB code simulates a multi-server queueing system — a mathematical model used to study waiting lines in contexts such as banks, hospitals, call centers, and supermarkets. The goal is to find the minimum number of servers (attendants, cashiers, agents, etc.) needed so that customers wait, on average, less than 5 minutes.

The simulation runs experiments with 1 server, then 2, then 3, and so on — up to 100 — until the target is met. For each scenario, it generates 1,000 customers with realistic random arrival and service times, then measures the average waiting time.

Quick Reference: Key Parameters

Parameter	Value	Meaning
numCustomers	1,000	Number of customers simulated in each experiment
meanArrival	0.1 min	Average time between consecutive customer arrivals (10 customers/min)
meanService	2 min	Average time to serve one customer
Target wait	< 5 min	Stopping criterion: simulation ends when this threshold is met

2. Block-by-Block Explanation

Block 1 — Problem Parameters

```
numCustomers = 1000;  
meanArrival = 0.1;    % minutes between arrivals  
meanService = 2;      % minutes per customer
```

This block defines the rules of the simulated world. These are the only numbers you would need to change to model a different real-world scenario.

- **numCustomers = 1000** — how many customers pass through the system in each simulation run.
- **meanArrival = 0.1** — on average, one customer arrives every 0.1 minute, meaning 10 customers per minute.
- **meanService = 2** — each customer requires an average of 2 minutes of service.

 **Analogy:** Think of a busy bank branch where customers arrive very frequently, but each transaction takes a couple of minutes — a combination that naturally creates a queue without enough tellers.

Block 2 — Exponential Distribution (Random Times)

```
interArrival = -meanArrival * log(rand(numCustomers,1));  
serviceTime = -meanService * log(rand(numCustomers,1));
```

This is the mathematical core of the simulation. To model the unpredictable behaviour of real customers, the code generates random numbers following an exponential distribution.

- **rand()** generates 1,000 random numbers uniformly distributed between 0 and 1.
- The formula **-mean × log(rand)** transforms those numbers into an exponential distribution — the standard mathematical model for inter-arrival and service times in queueing theory.
- The result: some customers arrive almost simultaneously, others with a longer gap — just like in real life.

 **Analogy:** Imagine scheduling bus arrivals: buses come on average every 10 minutes, but sometimes two arrive close together and sometimes there is a long gap. The exponential distribution captures exactly that variability.

Block 3 — Arrival Time Calculation

```
arrivalTime = cumsum(interArrival);  
arrivalTime = arrivalTime - arrivalTime(1);
```

The inter-arrival times generated above are intervals between consecutive customers. To know the actual clock time when each customer arrives, those intervals must be accumulated.

- **cumsum** (cumulative sum) adds each interval to all previous ones: customer 1 arrives at 0.08 min, customer 2 at $0.08 + 0.12 = 0.20$ min, and so on.
- The second line subtracts the first value so the simulation starts at time zero (i.e., the first customer arrives at $t = 0$).

💡 **Analogy:** *If trains depart with intervals of 5, 8, and 3 minutes, they leave at minutes 5, 13, and 16. cumsum does exactly that arithmetic automatically.*

Block 4 — Main Simulation Loop

```
for servers = 1:100
    serverFinish = zeros(servers, 1);

    for i = 1:numCustomers
        [nextFreeTime, serverIndex] = min(serverFinish);
        waitingTime(i) = max(0, nextFreeTime - arrivalTime(i));
        startService = max(arrivalTime(i), nextFreeTime);
        serverFinish(serverIndex) = startService + serviceTime(i);
    end
end
```

This is the heart of the simulation. The outer loop tests different numbers of servers (1, 2, 3 ... 100). For each configuration, the inner loop processes all 1,000 customers one by one.

- **serverFinish** is a list that tracks when each server will finish their current customer. At the start of each scenario, all servers are free (zeros).

For each customer, the code performs four steps:

- **min(serverFinish)** — finds the server who will become free soonest (the shortest queue logic).
- **waitingTime** — if that server is still busy when the customer arrives, the customer waits; otherwise the wait is 0.
- **startService** — service begins either when the customer arrives or when the server is free, whichever is later.
- **serverFinish** is then updated to reflect when that server will next be available.

💡 **Analogy:** *Picture a supermarket: you scan all checkout lanes and join the one that will clear first. If the cashier is still busy when you arrive, you wait; if the lane is empty, you go straight to the counter. The code replicates this decision for every customer.*

Block 5 — Stopping Criterion

```
averageWait = mean(waitingTime);
```

```

if averageWait < 5
    disp(['Optimal number of servers: ' num2str(servers)]);
    break;
end

```

After simulating all 1,000 customers, the code computes the mean waiting time. If it falls below 5 minutes, the current number of servers is declared optimal and the loop stops (break). Otherwise, one more server is added and the whole process repeats.

This logic ensures the model finds the smallest number of servers that satisfies the service-level target — avoiding both under-staffing (long queues) and over-staffing (unnecessary cost).

3. How the Pieces Fit Together

Step	What the code does	Real-world equivalent
Define parameters	Sets arrival rate, service time, and customer count	Describing how busy a branch is
Generate random times	Creates realistic, variable inter-arrival and service durations	Observing actual customer behaviour
Calculate arrival times	Converts intervals into clock times using cumsum	Logging each customer's entry timestamp
Simulate service	Assigns customers to the first available server, records wait	Running a shift at the service desk
Evaluate & decide	Checks if average wait < 5 min; stops or adds a server	Manager deciding whether to open another counter

4. Key Concepts to Remember

The following three ideas are the most important to understand and communicate:

- **Why the exponential distribution?** It is the standard mathematical model for random, memoryless events such as customer arrivals and service completions — the same assumption used in classical M/M/c queueing theory.
- **Why 1,000 customers?** A larger sample reduces statistical noise. With only 10 customers, luck dominates the result; with 1,000, the average is reliable and representative.

- **Why `min(serverFinish)`?** Directing each customer to the server who will be free soonest is the optimal greedy strategy — it minimises idle time and mirrors how a smart queue manager would operate.

5. Possible Modifications

Once the base code is understood, it is easy to adapt it to other scenarios by changing the parameters:

Change	How to modify
Different arrival rate	Adjust <code>meanArrival</code> (e.g., 0.5 for 2 customers/min)
Different service time	Adjust <code>meanService</code> (e.g., 0.5 for fast service)
Different wait threshold	Change the 5-minute limit in the if condition
More customers	Increase <code>numCustomers</code> for higher statistical confidence
Different distribution	Replace the exponential formula with another (e.g., normal)
